

EC 551 – Advanced Digital Design with Verilog and FPGAs

Lecture 7 Behavioral Modeling

Prof. Douglas Densmore

Announcements

- Thursday 10/1 presentations! Order to be posted to Blackboard later today/tomorrow.
- Everyone group should have posted groups in Piazza (EOD today)!
- HW2 out Thursday 10/1
- HW1 due Thursday 10/1 – Gradescope coming!

Behavioral Models



- ❑ Behavioral models are abstract descriptions of functionality.
 - Describes **what** the circuit will do, not **how** to build it in hardware
- ❑ Widely used for quick development of model
- ❑ Follow by synthesis
- ❑ We'll consider two types:
 - Continuous assignment (Boolean equations)
 - Cyclic behavior (more general, e.g. algorithms)

Data Types



- ❑ Two families of data types for variables:
 - Nets: **wire**, tri, wand, triand, wor, trior, supply0, supply1
 - Registers: **reg**, **integer**, real, time, realtime
 - Nets establish structural connectivity
 - Register variables act as storage containers for the waveform of a signal
- ❑ Default size of a **net** or **reg** variable is a signal bit
- ❑ An **integer** is stored at a minimum of 32 bits
- ❑ **time** is stored as 64 bit integer
- ❑ **real** is stored as a real number
- ❑ **realtime** stores the value of time as a real number

Combinational Behavior



□ Combinational behavior

- Description of desired behavior of combinational circuit without creating circuit itself
- e.g., $F = c' * (h + p)$ can be described as equation rather than circuit
- HDLs support description of combinational behavior

Continuous Assignment

- ❑ Implies structural hardware
assign <LHS> = <RHS expression>;
- ❑ Example
 wire out, a, b;
 assign out = a & b;
- ❑ If RHS result changes, LHS is updated with new value
 - Constantly operating (“continuous”!)
 - It’s ***hardware!***
- ❑ Used to model combinational logic and latches

Continuous Assignment LHS

- Can assign values to:
 - Scalar nets
 - Vector nets
 - Single bits of vector nets
 - Part-selects of vector nets
 - Concatenation of any of the above

- Examples:

assign out[7:4] = a[3:0] | b[7:4];

assign val[3] = c & d;

assign {a, b} = stimulus[15:0];

Continuous Assignment RHS

- Use operators:
 - Arithmetic, Logical, Relational, Equality, Bitwise, Reduction, Shift, Concatenation, Replication, Conditional
 - Same set as used in Behavioral Verilog
- - assign** a = stimulus[16:9];
 - assign** b = stimulus[8:1];
 - assign** cin = stimulus[0];

Simple Behavioral Model

```
module and_gate (out, in1, in2);  
  input          in1, in2;  
  output         out;
```

Output port of a *primitive* must
be first in the list of ports

```
    assign out = in1 & in2;
```

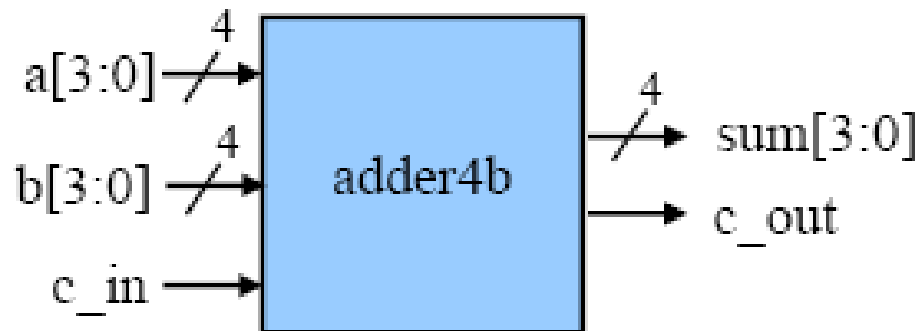
Restriction does not apply to
modules

```
endmodule
```

When is this evaluated?

Example: adder4b

```
module adder4b (sum, c_out, a, b, c_in);  
    input      [3:0]  a, b;  
    input      c_in;  
    output     [3:0]  sum;  
    output     c_out;  
    assign {c_out, sum} = a + b + c_in;  
endmodule
```



Simple behavioral model

□ always block

```
module xor_gate (z, a, b);  
  input          a, b;  
  output         z;  
  reg            z;  
  
  always @(a or b) begin  
    z = a ^ b;  
  end  
  
endmodule
```

Sensitivity list

specifies when block is executed
ie. triggered by which signals

Simple behavioral model

Short-hand notation

Continuous assignment

```
module xor_gate (z, a, b);  
    input        a, b;  
    output       z;  
    reg          z;  
  
    assign z = a ^ b;  
  
endmodule
```

simulation register -
keeps track of
value of signal

- Notes:

- Target output of procedural assignments must be of type `reg`
(not a real register) ISE-?
- Unlike `wire` types where the target output of an assignment may be continuously updated, a `reg` type retains its value until a new value is assigned (the assigning statement is executed).

Verilog Continuous Assignment

- Assignment is continuously evaluated
- `assign` corresponds to a connection or a simple component with the described function
- Target is *NEVER* a (real) reg variable
- Dataflow style

assign A = X | (Y & ~Z);

← use of Boolean operators
(~ for bit-wise, ! for logical negation)

assign B[3:0] = 4'b01XX;

← bits can take on four values
(0, 1, X, Z)

assign C[15:0] = 4'h00ff;

← variables can be n-bits wide
(MSB:LSB)

assign #3 {Cout, S[3:0]} = A[3:0] + B[3:0] + Cin;

← use of arithmetic operator

← multiple assignment (concatenation)

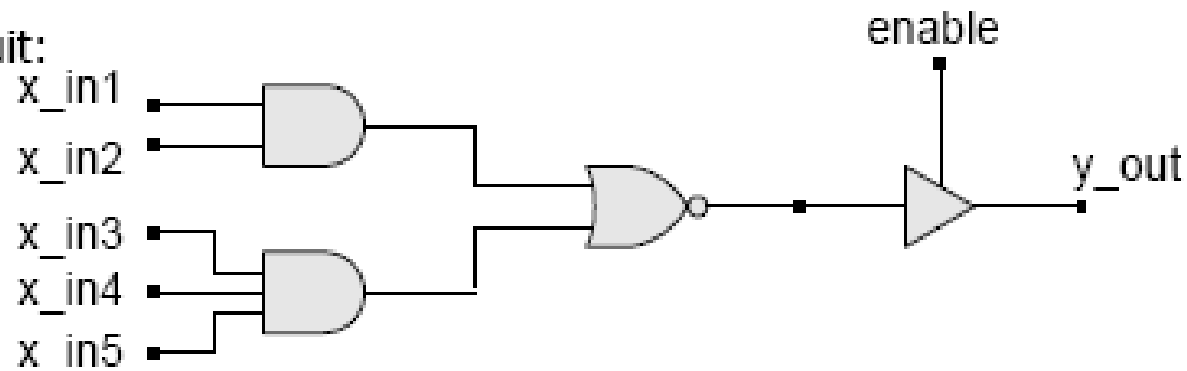
← delay of performing computation, only used by simulator, not synthesis

AOI circuit with *enable*

```
module AOI_5_CA1 (y_out, x_in1, x_in2, x_in3, x_in4, x_in5, enable);  
  input          x_in1, x_in2, x_in3, x_in4, x_in5, enable;  
  output         y_out;  
  assign y_out = enable ? ~((x_in1 & x_in2) | (x_in3 & x_in4 & x_in5)) : 1'bz;  
endmodule
```

- ❑ The *conditional operator* (*? :*) acts like a software if-then-else switch that selects between two expressions.
- ❑ Must provide both expressions

Equivalent circuit:



Behavioral 4-to1 mux

//4-input mux behavioral description

```
module mux4 (in0, in1, in2, in3, select, out);
```

```
    input in0,in1,in2,in3;
```

```
    input [1:0] select;
```

```
    output      out;
```

```
    reg        out;
```

```
    always @ (in0 in1 in2 in3 select)
```

```
        case (select)
```

```
            2'b00: out=in0;
```

```
            2'b01: out=in1;
```

```
            2'b10: out=in2;
```

```
            2'b11: out=in3;
```

```
        endcase
```

```
endmodule // mux4
```

□ Notes:

- No instantiation
- Case construct equivalent to nested if constructs.
- **Definition:** A structural description is one where the function of the module is defined by the instantiation and interconnection of sub-modules.
- A behavioral description uses higher level language constructs and operators.
- Verilog allows modules to mix both behavioral constructs and sub-module instantiation.

Mixed Structural/Behavioral Model

□ Example 4-bit ripple adder

```
module full_addr (S, Cout, A, B, Cin );  
    input      A, B, Cin;  
    output     S, Cout;  
  
    assign {Cout, S} = A + B + Cin;  
endmodule
```

Behavior

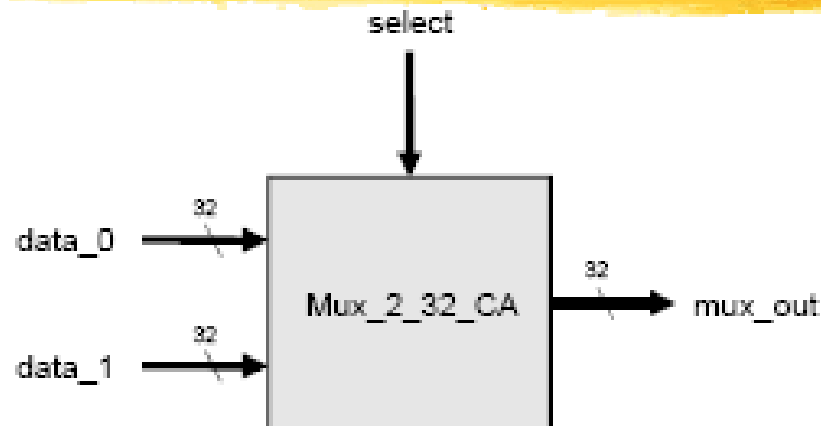
```
module adder4 (S, Cout, A, B, Cin);  
    input  [3:0] A, B;  
    input      Cin;  
    output [3:0] S;  
    output     Cout;  
    wire      C1, C2, C3;  
  
    full_addr fa0 (S[0], C1,  A[0], B[0], Cin);  
    full_addr fa1 (S[1], C2,  A[1], B[1], C1);  
    full_addr fa2 (S[2], C3,  A[2], B[2], C2);  
    full_addr fa3 (S[3], Cout, A[3], B[3], C3);  
endmodule
```

Structural

Implicit Continuous Assignment

```
module AOI_5_CA2 (y_out, x_in1, x_in2, x_in3, x_in4, x_in5, enable);  
  input          x_in1, x_in2, x_in3, x_in4, x_in5, enable;  
  output         y_out;  
  wire y_out = enable ? ~((x_in1 & x_in2) | (x_in3 & x_in4 & x_in5)) : 1'bz;  
  
endmodule
```

A two-channel mux with 32-bit datapath



```
module Mux_2_32_CA ( mux_out, data_1, data_0, select);  
  parameter word_size = 32;  
  output      [word_size -1: 0] mux_out;  
  input       [word_size-1: 0] data_1, data_0;  
  input                               select;  
  assign mux_out = select ? data_1 : data_0;  
endmodule
```

Parameterized Modules: Example

□ N-bit 2-1 multiplexer (parameterized bitwidth)

```
module mux2(sel, a, b, y);  
    parameter bitwidth=32;  
    input sel;  
    input  [bitwidth-1:0] a, b;  
    output [bitwidth-1:0] y;  
  
    assign y = sel ? b : a;  
endmodule
```

Defines Parameter **bitwidth**
(default value:32)

Uses Parameter **bitwidth**
to set input, output **size**

□ Instantiations

```
mux2 #(16) my16bit_mux(s, a ,b, c);  
mux2 #(5)  my5bit_mux(s, d, e, f);  
mux2 #(32) my32bit_mux(s, g, h, i);  
mux2 myDefault32bit_mux(s, j, k, l);
```

← 16-bit mux

← 5-bit mux

← 32-bit mux

← 32-bit mux (default)

Simple behavioral model. Delay

```
module xor_gate (a, b, z);  
    input        a, b;  
    output       z;  
    reg          z;  
  
    assign #6 z = a ^ b;  
  
endmodule
```

```
module xor_gate (a, b, z)  
    input        a, b;  
    output       z;  
    reg          z;  
  
    always @(a or b) begin  
        #6 z = a ^ b;  
    end  
endmodule
```

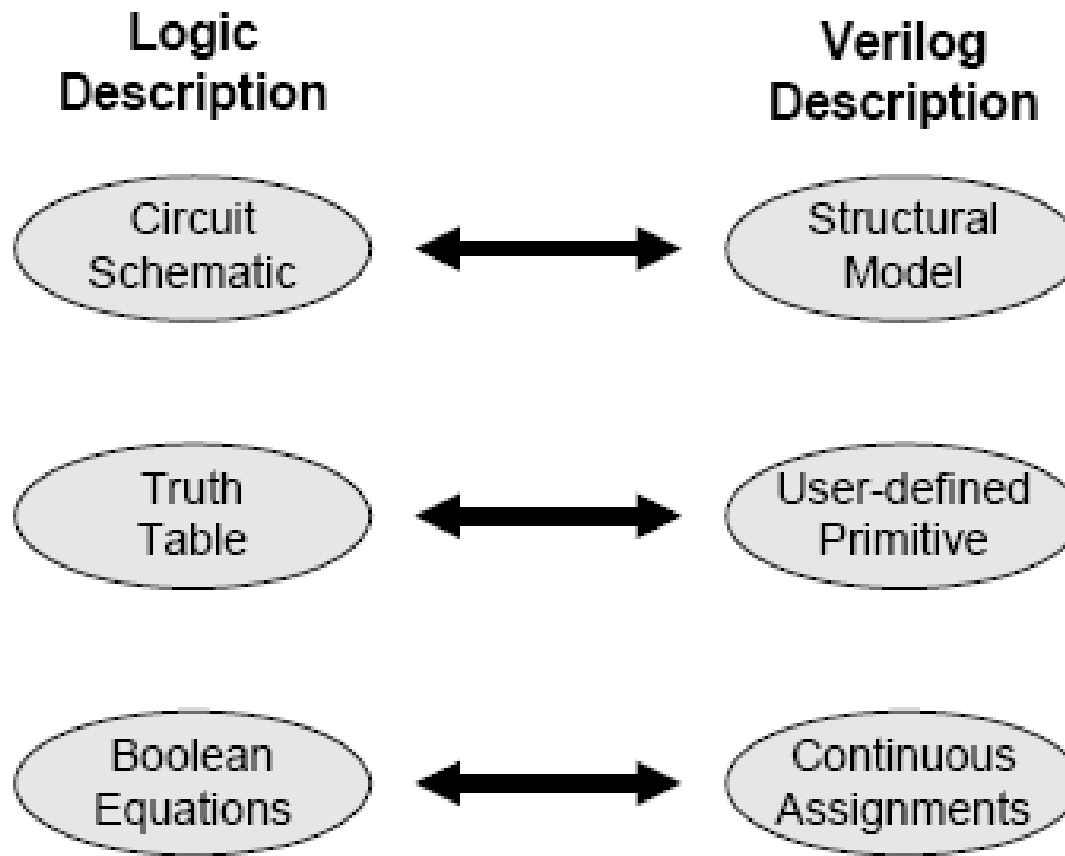
delay from input change
to output change

Multiple Continuous Assignments

- Multiple continuous assignments are **active concurrently**

```
module compare_2_CA0 (A_lt_B, A_gt_B, A_eq_B, A1, A0, B1, B0);  
  input  A1, A0, B1, B0;  
  output      A_lt_B, A_gt_B, A_eq_B;  
  
  assign A_lt_B = (~A1) & B1 | (~A1) & (~A0) & B0 | (~A0) & B1 & B0;  
  assign A_gt_B = A1 & (~B1) | A0 & (~B1) & (~B0) | A1 & A0 & (~B0);  
  assign A_eq_B = (~A1) & (~A0) & (~B1) & (~B0) | (~A1) & A0 & (~B1) & B0  
  | A1 & A0 & B1 & B0 | A1 & (~A0) & B1 & (~B0);  
  
endmodule
```

Review of Modeling Styles for Combinational Logic



Behavioral Verilog



- ❑ Use procedural blocks: **initial**, **always**
- ❑ These blocks contain series of statements
 - Abstract – works *somewhat* like software
 - Be careful to still remember it's hardware!
- ❑ Parallel operation across blocks
 - All blocks in a module operate simultaneously
- ❑ Sequential or parallel operation within blocks
 - Depends on the way the block is written (= vs.<=)
 - Discuss this later
- ❑ LHS of assignments are variables (**reg**)

Types of Blocks



□ initial

- Behavioral block operates ONCE
- Starts at time 0 (beginning of operation)
- Useful for testbenches
- Can sometimes provide initialization of memories/FFs
 - Often better to use "reset" signal
- Inappropriate for combinational logic
- Usually cannot be synthesized

□ always

- Behavioral block operates CONTINUOUSLY
- Can use a *trigger list* to control operation; @(a, b, c)

initial Blocks

```
`timescale 1ns /1ns
module t_full_adder;
  reg [3:0] stim;
```

*all initial blocks
start at time 0*

```
  wire s, c;

  // instantiate UUT
  full_adder(sum, carry, stim[2], stim[1], stim[0]);
```

```
  // monitor statement is special - only needs to be made once,
  initial $monitor("%t: s=%b c=%b stim=%b", $time, s, c, stim[2:0]);
```

```
  // tell our simulation when to stop
  initial #50 $stop;
```

single-statement block

```
  initial begin // stimulus generation
    for (stim = 4'd0; stim < 4'd8; stim = stim + 1) begin
      #5,
```

*multi-statement
block enclosed by
begin and **end***

```
    end
  end
endmodule
```

always Blocks

- ❑ Operates continuously or on a trigger list
- ❑ Can be used with **initial** blocks
- ❑ Cannot “nest” initial or always blocks
- ❑ Useful example of continuous always block:

```
reg clock;  
initial clock = 1'b0;  
always #10 clock = ~clock;
```

- ❑ Clock generator goes in the testbench

always blocks with sensitivity (trigger) lists

- Conditionally “execute” inside of **always** block

- Always block continuously operating
- If sensitivity list present, continuously checking triggers
- Any change on trigger (sensitivity) list, triggers block

always @(a, b, c) **begin**

...

end

- Sounds like software! It isn't!

- This is how the *simulator* treats it
- The hardware has the same resulting operation, but...
 - See examples in later slides to see what is actually created

Event or



- ❑ Original way to specify sensitivity list
always @ (X1 or X2 or X3)
- ❑ In Verilog 2001 can use , instead of **or**
always @ (X1, X2, X3)

Latched and Level-Sensitive Behavior

- ❑ Avoid explicit or implicit structural feedback
- ❑ It simulates but won't synthesize
- ❑ Timing analyzers won't work either

Example

assign q = set ~& qbar;
assign qbar = rst ~& q;

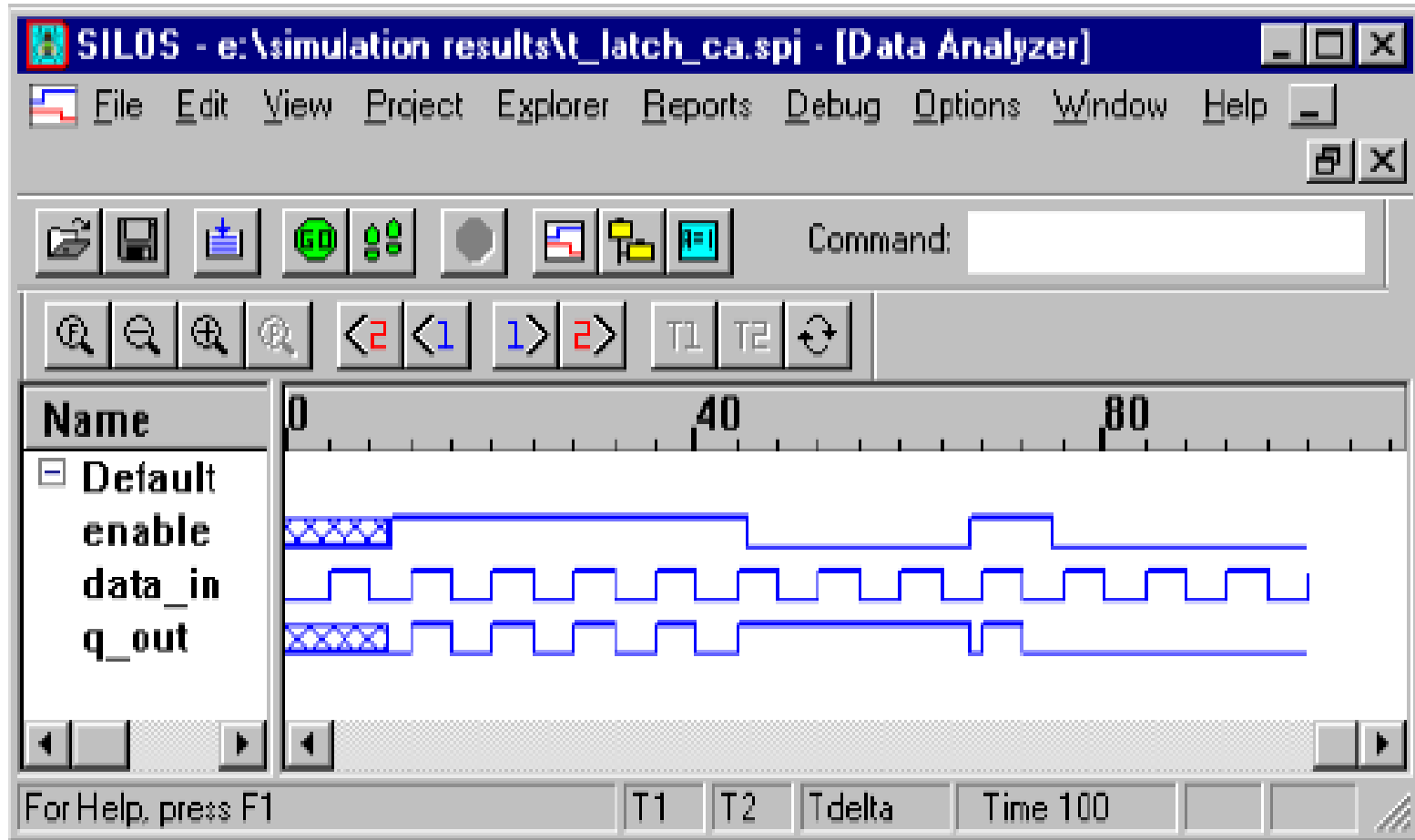
Recommended Style for Transparent Latch

- ❑ Use a **continuous assignment with feedback** to model a latch
- ❑ Synthesis tools understand this model (Why?)

Example

```
module Latch_CA (q_out, data_in, enable);  
  output      q_out;  
  input       data_in, enable;  
  assign q_out = enable ? data_in : q_out;  
endmodule
```

Simulation results:

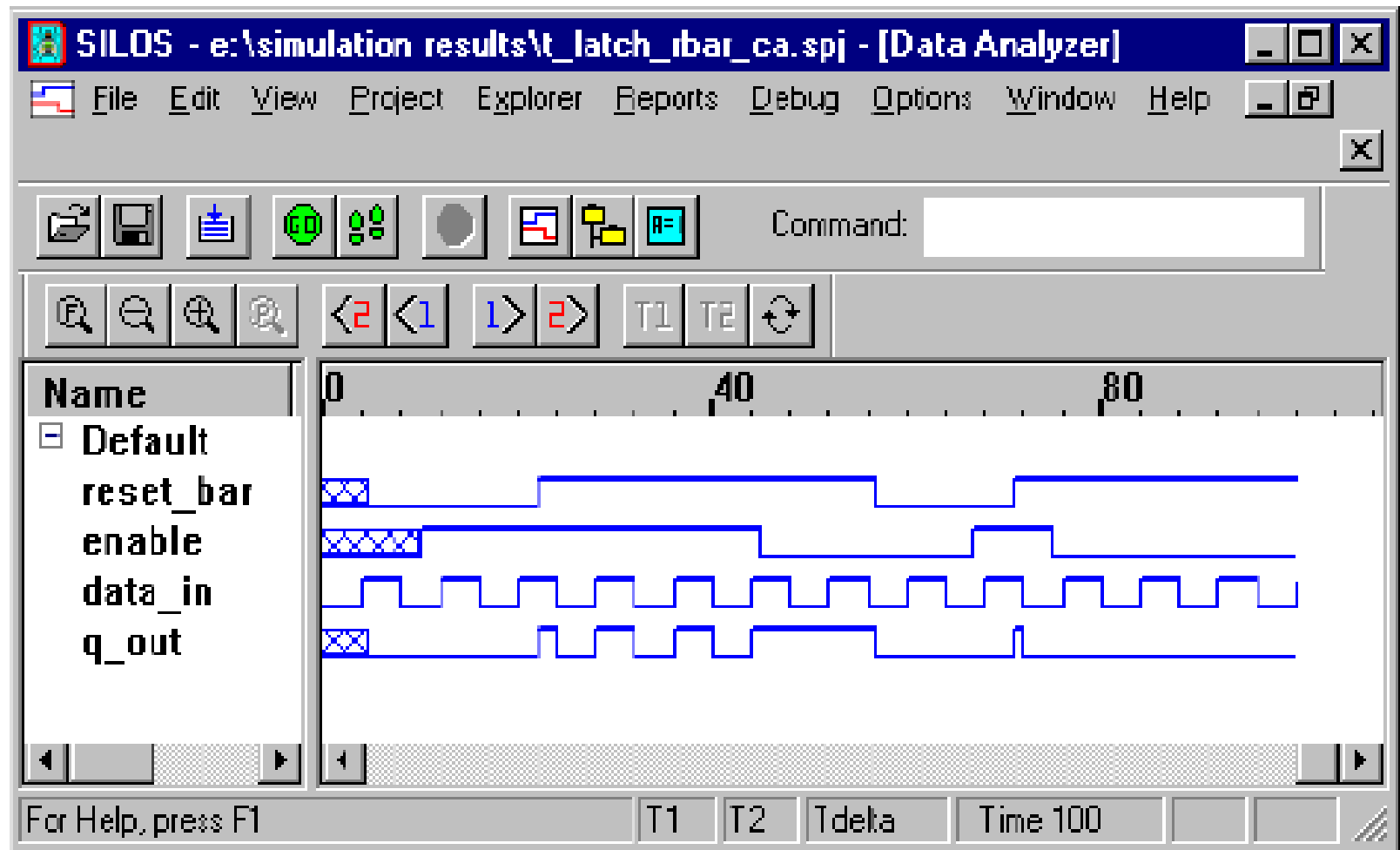


Example: Latch with Active-Low Reset

- Example transparent latch with active-low reset (nested conditional operators)

```
module Latch_Rbar_CA (q_out, data_in, enable, reset);  
  output      q_out;  
  input       data_in, enable, reset;  
  assign q_out = !reset ? 0 : enable ? data_in : q_out;  
endmodule
```

Simulation results:



Abstract Modeling with Cyclic Behaviors



- ❑ Continuous assignments statements (assign) are limited to modeling level-sensitive behavior (not for flip-flops)
- ❑ Cyclic behaviors (always) are abstract - they assign values to register variables to describe the behavior of hardware
- ❑ Model level-sensitive and edge-sensitive behavior

Combinational vs. Sequential

□ Combinational

- Not edge-triggered
- All “inputs” (RHS nets/variables) are triggers
- Does not depend on clock

□ Sequential

- Edge-triggered by clock signal
- Only clock (and possibly reset) appear in trigger list
- Can include combinational logic that feeds a FF or register

“Complete” Assignments

- ❑ If an `always` block executes, and a variable is *not* assigned
 - Variable keeps its old value (think implicit state!)
 - *NOT* combinational logic $\Rightarrow$ latch is inserted (implied memory)
 - This is usually *not* what you want: dangerous for the novice!
- ❑ **Any variable assigned in an `always` block should be assigned for any (and every!) execution of the block**

Incomplete Triggers

- ❑ Leaving out an input trigger usually results in a sequential circuit
- ❑ Example: The output of this "and" gate depends on the input history

```
module and_gate (out, in1, in2);  
    input        in1, in2;  
    output       out;  
    reg          out;  
  
    always @(in1) begin  
        out = in1 & in2;  
    end  
  
endmodule
```

Verilog always statement

- Use always to wait for clock edge:

```
always @(posedge clock) // start execution at the clock
    edge
begin
    // insert combinational logic here
end
```

Flip-flop in Verilog

- Use always block's sensitivity list to wait for clock edge

```
module dff (clk, d, q);  
  
    input  clk, d;  
    output q;  
    reg    q;  
  
    always @(posedge clk)  
        q = d;  
  
endmodule
```

Incorrect Flip-flop in Verilog

- ❑ Use always block's sensitivity list to wait for clock to change

```
module dff (clk, d, q);  
  
    input  clk, d;  
    output q;  
    reg    q;  
  
    always @(clk)  
        q = d;  
  
endmodule
```

Not correct! Q will
change whenever the
clock changes, not
just on the edge.



Example: D-type Flip-Flop with set/reset

```
module df_behav (q, q_bar, data, set, reset, clk);  
input          data, set, clk, reset;  
output         q, q_bar;  
reg            q;  
assign q_bar = ~ q;  
always @ (posedge clk) // Flip-flop with synchronous set/reset  
begin  
    if (reset == 0) q <= 0; // <= is the nonblocking assignment operator  
    else if (set == 0) q <= 1;  
    else q <= data;  
end  
endmodule
```

Procedural Assignments

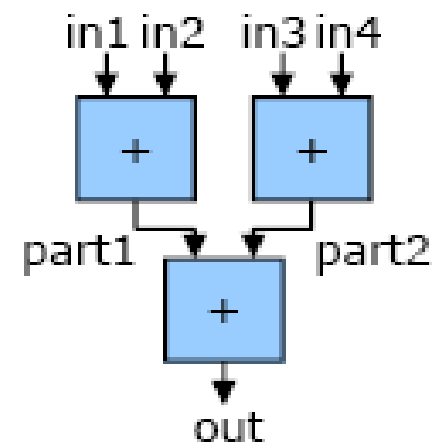
- ❑ Used within a behavioral block (initial, always)
- ❑ Types
 - = // blocking assignment
 - <= // non-blocking assignment
- ❑ Assignments to variables:
 - reg
 - integer
 - real
 - realtime
 - time

Blocking Assignments

- ❑ "Evaluated" sequentially
- ❑ Works a lot like software (**danger!**)
- ❑ Used for combinational logic

```
module addtree(output reg [9:0] out,  
               input [7:0] in1, in2, in3, in4);
```

```
  reg [8:0] part1, part2;  
  always @(in1, in2, in3, in4) begin  
    part1 = in1 + in2;  
    part2 = in3 + in4;  
    out = part1 + part2;  
  end  
endmodule
```



Blocking and Non-Blocking Assignments

- Blocking (sequential) assignments ($X=A$)
 - completes the assignment **before continuing on to next statement**
- Non-blocking (delayed) assignments ($X<=A$)
 - completes in zero time and **doesn't change the value of the target until a blocking point (delay/wait) is encountered**
- Example: swap

```
always @(posedge CLK)
begin
    temp = B;
    B = A;
    A = temp;
end
```

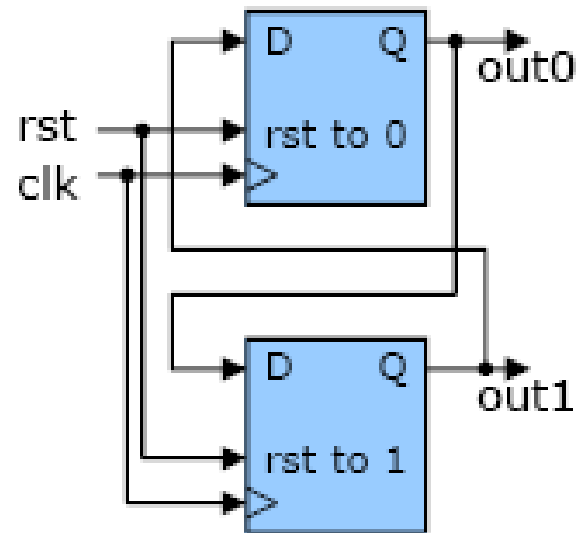
```
always @(posedge CLK)
begin
    A <= B;
    B <= A;
end
```

Non-Blocking Assignments

- ❑ "Updated" simultaneously if no delays given
- ❑ Used for sequential logic

```
module swap(output reg out0, out1, input rst, clk);
```

```
  always @(posedge clk) begin
    if (rst) begin
      out0 <= 1'b0;
      out1 <= 1'b1;
    end
    else begin
      out0 <= out1;
      out1 <= out0;
    end
  end
end
endmodule
```



More Flip-flops

□ Synchronous/asynchronous reset/set

- **Synchronous** - single thread that waits for the clock
- **Asynchronous** - three parallel threads – only one of which waits for the clock

Synchronous

```
module dff (clk, s, r, d, q);  
  input  clk, s, r, d;  
  output q;  
  reg    q;  
  
  always @(posedge clk)  
    if (r) q = 1'b0;  
    else if (s) q = 1'b1;  
    else q = d;  
  
endmodule
```

Asynchronous

```
module dff (clk, s, r, d, q);  
  input  clk, s, r, d;  
  output q;  
  reg    q;  
  
  always @(posedge r)  
    q = 1'b0;  
  always @(posedge s)  
    q = 1'b1;  
  always @(posedge clk)  
    q = d;  
  
endmodule
```

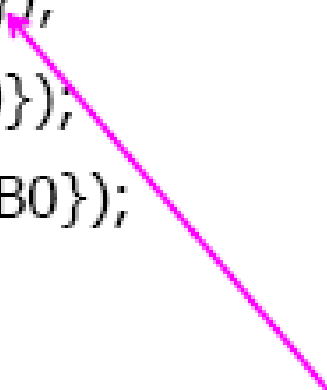
Example: Transparent Latch (Cyclic Behavior)

```
module tr_latch (q_out, enable, data);  
  output q_out;  
  input enable, data;  
  reg q_out;  
  always @ (enable or data)  
  begin  
    if (enable) q_out = data;  
  end  
endmodule
```

Alternative Behavioral Models (Two-bit comparator)

```
module compare_2_CA1 (A_lt_B, A_gt_B, A_eq_B, A1, A0,  
    B1, B0);  
  
input          A1, A0, B1, B0;  
  
output         A_lt_B, A_gt_B, A_eq_B;  
  
assign    A_lt_B = ({A1,A0} < {B1,B0});  
  
assign    A_gt_B = ({A1,A0} > {B1,B0});  
  
assign    A_eq_B = ({A1,A0} == {B1,B0});  
  
endmodule
```

Concatenation operator



Alternative Behavioral Models (Two-bit comparator with 2-bit vectors)

Example (Clarity!)

```
module compare_2_CA1 (A_lt_B, A_gt_B, A_eq_B, A, B);  
  input [1: 0]  A, B;  
  output          A_lt_B, A_gt_B, A_eq_B;  
  assign A_lt_B = (A < B); // The RHS expression is true (1) or false (0)  
  assign      A_gt_B = (A > B);  
  assign      A_eq_B = (A == B);  
  endmodule
```

Alternative Behavioral Models (Two-bit comparator with 32-bit vectors)

Example (Parameterized and reusable model)

```
module compare_32_CA (A_gt_B, A_lt_B, A_eq_B, A, B);  
    parameter          word_size = 32;  
    input               [word_size-1: 0]      A, B;  
    output              A_gt_B, A_lt_B, A_eq_B;  
    assign              A_gt_B = (A > B), // Note: list of multiple assignments  
                  A_lt_B = (A < B),  
                  A_eq_B = (A == B);  
endmodule
```

compare_2_RTL executes and updates the outputs whenever a bit of either datapath changes

```
module compare_2_RTL (A_lt_B, A_gt_B, A_eq_B, A1, A0, B1, B0);  
  input      A1, A0, B1, B0;  
  output     A_lt_B, A_gt_B, A_eq_B;  
  reg        A_lt_B, A_gt_B, A_eq_B;  
  always @ (A0 or A1 or B0 or B1) begin  
    A_lt_B = ({A1,A0} < {B1,B0});  
    A_gt_B = ({A1,A0} > {B1,B0});  
    A_eq_B = ({A1,A0} == {B1,B0});  
  end  
endmodule
```

Modeling Trap

- ❑ The order of execution of ordinary **procedural assignments** (blocked (or sequential) assignments ($X=A$)) in a cyclic behavior **may depend on the order** in which the statements are listed (it completes the assignment before continuing on to next statement)
- ❑ A **procedural assignment** cannot execute until the previous statement executes
- ❑ Because there are **no dependencies** between the variables of the LHS of the three procedural assignments in *compare_2_RTL* the order in which the assignment are listed does not affect the outcome
- ❑ *In General: Failure to appreciate the effects of expression substitution can lead to incorrect models*

Register-transfer-level (RTL) Assignment

- **Non-blocking assignment** is also known as an **RTL assignment**
 - if used in an always block triggered by a clock edge
 - **all flip-flops change together**

```
// B,C,D all get the value of A
always @(posedge clk)
begin
    B = A;
    C = B;
    D = C;
end
```

Blocking assignment

Non-blocking assignment

```
// implements a shift register too
always @(posedge clk)
begin
    B <= A;
    C <= B;
    D <= C;
end
```

Why Not '=' In Sequential?

- Yes, it can “work”, but...
 - <= models pipeline stages better
 - = can cause problems if multiple always blocks
 - Order of statements is important with –
- Use the style guidelines given!
 - <= for sequential block
 - = for combinational block
 - Don't mix in same block!

Shift register (with blocking assignment)

```
module shift4bit (clk, in, out1, out2, out3, out4)
    input          clk, in;
    output         out1, out2, out3, out4;
    reg           out1, out2, out3, out4;

    always @(posedge clk) begin
        out4 = out3; // the order is important
        out3 = out2;
        out2 = out1;
        out1 = in;    end
endmodule
```

Shift register (with non-blocking assignment)

```
module shift4bit (clk, in, out1, out2, out3, out4)
    input          clk, in;
    output         out1, out2, out3, out4;
    reg            out1, out2, out3, out4;

    always @(posedge clk) begin
        out1 <= in;
        out2 <= out1;
        out3 <= out2;
        out4 <= out3;
    end

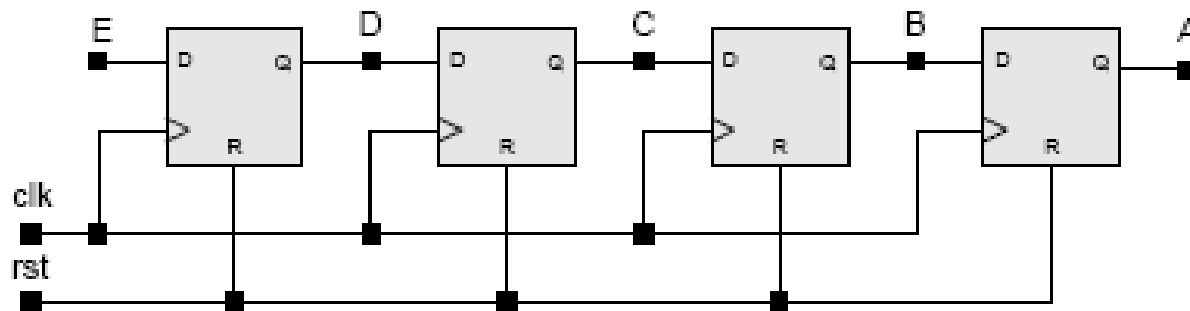
endmodule
```

Note that now the order of the statements does not matter.

Another shift register with reset (with blocking assignment)

```
module shiftreg_PA (E, A, B, C, D, clk, rst);  
    output      A;  
    input       E;  
    input       clk, rst;  
    reg A, B, C, D;  
    always @ (posedge clk or posedge rst) begin  
        if (rst) begin A = 0; B = 0; C = 0; D = 0; end  
        else begin  
            A = B;  
            B = C;  
            C = D;  
            D = E;  
        end  
    end  
endmodule
```

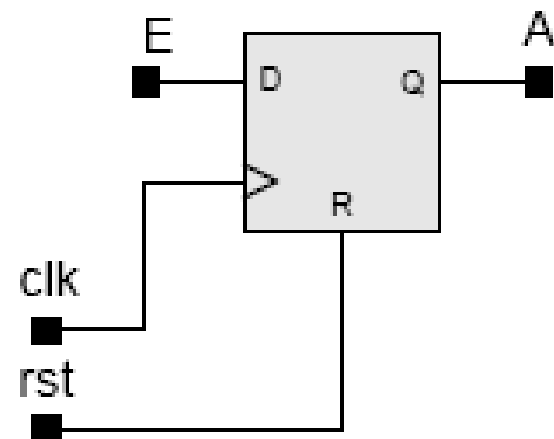
Result of synthesis:



Reverse the order of the statements:

```
module shiftreg_PA_rev (A, E, clk, rst);  
    output      A;  
    input       E;  
    input       clk, rst;  
    reg A, B, C, D;  
    always @ (posedge clk or posedge rst) begin  
        if (rst) begin A = 0; B = 0; C = 0; D = 0; end  
        else begin  
            D = E;  
            C = D;  
            B = C;  
            A = B;  
        end  
    end  
end  
endmodule
```

Result of synthesis:



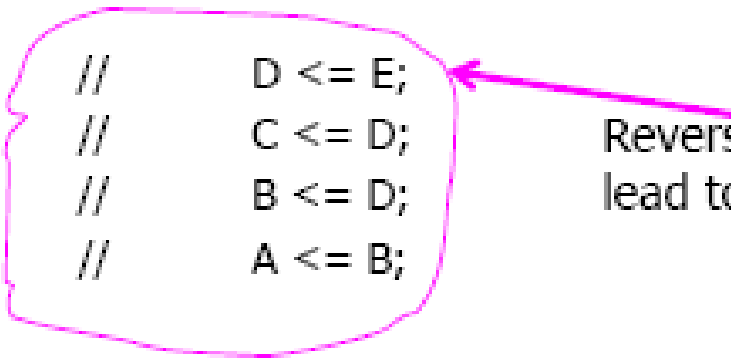
Nonblocking Assignment Operator and Concurrent Assignments



- ❑ Non-blocking assignment ($B \leftarrow A$) statements execute *concurrently* (in parallel) rather than sequentially
- ❑ The order in which non-blocking assignments are listed has no effect.
- ❑ Mechanism: the RHS of the assignments are sampled (held in memory), then assignments are updated
- ❑ Assignments are based on values held by RHS before the statements execute
- ❑ Result: No dependency between statements

Example: Shift Register (nonblocking with reset)

```
module shiftreg_nb (A, E, clk, rst);  
    output      A;  
    input       E;  
    input       clk, rst;  
    reg         A, B, C, D;  
    always @ (posedge clk or posedge rst) begin  
        if (rst) begin A <= 0; B <= 0; C <= 0; D <= 0; end  
        else begin  
            A <= B;  
            B <= C;  
            C <= D;  
            D <= E;  
        end  
    end  
endmodule
```



```
//      D <= E;  
//      C <= D;  
//      B <= C;  
//      A <= B;
```

Reversed order would
lead to the same results

Functions and Tasks

- Improve the clarity, portability and maintainability by **encapsulating** and organizing code into tasks and functions
 - Tasks create a hierarchical organization of the procedural statements
 - Task are more general than functions and may contain timing control
 - Function substitute for an expression and are equivalent to combinational logic
- Tasks
 - Tasks declared within a module
 - Task may be referenced only from within a cyclic or single-pass behavior
 - Task must be named and may include declaration of any number of combination of : parameter, input, output, etc.
 - All declarations of variable are local to the task
 - Task that are to be synthesized may contain event-control operators[@], but not delay-control operators[#]

Tasks

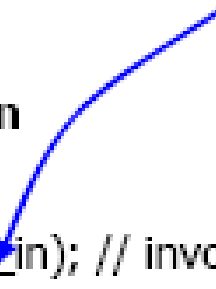


- ❑ Declared within a module – only referenced within a behavior
- ❑ Tasks provide the ability to
 - Execute common procedures from multiple places
 - Divide large procedures into smaller ones
- ❑ Tasks are not limited to combinational logic
 - Can have time-controlling statements (@, #, wait)
 - Some of this better for testbenches
- ❑ Can have multiple outputs, inout ports
- ❑ Local variables can be declared & used

Task Example [Part 1]

```
module adder_task (c_out, sum, clk, reset, c_in, data_a, data_b, clk);  
  output reg    [3: 0]    sum;  
  output reg          c_out;  
  input  [3: 0]    data_a, data_b;  
  input          clk, reset, c_in;
```

*this is NOT conditionally
"creating" hardware!*



```
  always @(posedge clk or posedge reset) begin  
    if (reset) {c_out, sum} <= 0;  
    else add_values (c_out, sum, data_a, data_b, c_in); // invoke task  
  end  
  // Continued on next slide
```

Task Example [Part 2]

// Continued from previous slide

task add_values; // task declaration

output reg	[3: 0]	sum;	} <i>task outputs</i>
output reg		c_out;	
input	[3: 0]	data_a, data_b;	} <i>task inputs</i>
input		c_in;	
{c_out, sum} <= data_a + (data_b + c_in);			

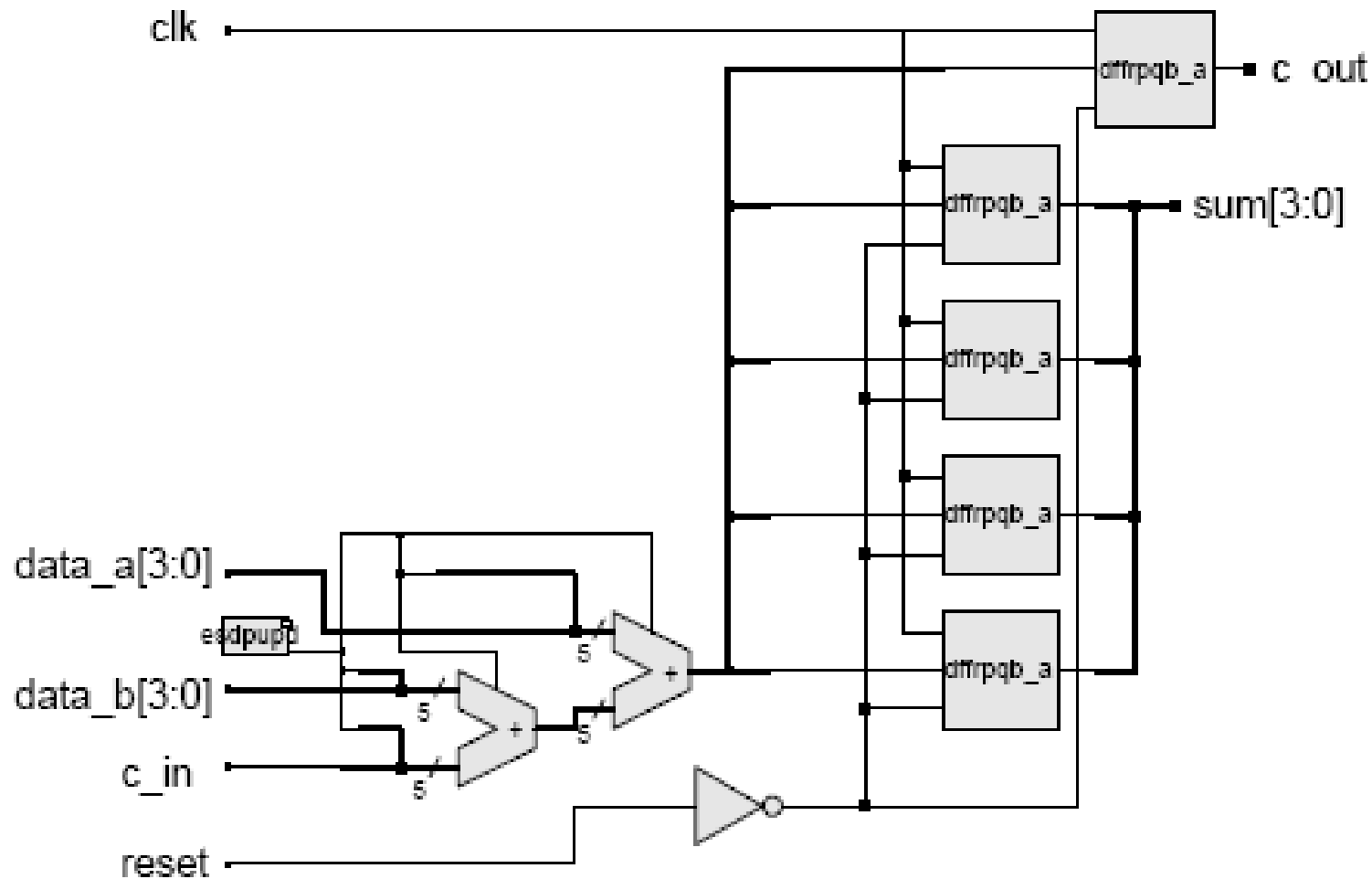
endtask

endmodule

- Could have instead specified inputs/outputs using a port list.

```
task add_values (output reg [3: 0] sum, output reg c_out,  
                input [3:0] data_a, data_b, input c_in);
```

Circuit synthesized from adder_task



Functions



- ❑ Declared and referenced within a module
- ❑ Used to implement **combinational behavior**
 - Contain no timing controls or tasks
- ❑ Inputs/outputs
 - Must have at least one input argument
 - Has only one output (no inouts)
 - Function name is implicitly declared return variable
 - Type and range of return value can be specified (1-bit wire is default)

Functions

- ❑ Functions are declared within a parent module and **can be referenced in any valid expression**, e.g. in RHS of a continuous-assignment statement
- ❑ **Functions may implement only combinational behavior**
 - they **may not contain** timing controls([#]) or **event control**[@], or **wait**
 - they may not invoke a task
 - they may call other functions , **but not recursively**
- ❑ Function may contain declarations of inputs and local variables
- ❑ **The value of a function is returned by its name**
 - Function may not have any declared outputs or inout ports
- ❑ Function must have at least one input argument
- ❑ **The execution of a function takes place in zero time in the same time step as calling expression is evaluated**
- ❑ The definition of the function **implicitly defines an internal register variable**

Function Example: Word Aligner

```
module word_aligner (word_out, word_in);  
    output      [7: 0]  word_out;  
    input       [7: 0]  word_in;  
    assign word_out = aligned_word(word_in); // invoke function  
  
    function      [7: 0]  aligned_word;      // function declaration  
        input      [7: 0]  word;            ← input to function  
        begin  
            aligned_word = word;  
            if (aligned_word != 0)  
                while (aligned_word[7] == 0) aligned_word = aligned_word << 1;  
        end  
    endfunction  
endmodule
```

size of return value

input to function

Function Example: Arithmetic Unit

```
module arithmetic_unit (result_1, result_2, operand_1, operand_2,);
```

```
output      [4: 0] result_1;
```

```
output      [3: 0] result_2;
```

```
input       [3: 0] operand_1, operand_2;
```

```
assign result_1 = sum_of_operands (operand_1, operand_2);
```

```
assign result_2 = larger_operand (operand_1, operand_2);
```

function call

```
function [4: 0] sum_of_operands(input [3: 0] operand_1, operand_2);
```

```
{ sum_of_operands = operand_1 + operand_2;
```

```
endfunction
```

function output

```
function [3: 0] larger_operand(input [3: 0] operand_1, operand_2);
```

```
{ larger_operand = (operand_1 >= operand_2) ? operand_1 : operand_2;
```

```
endfunction
```

```
endmodule
```

function inputs

Next Time

- Presentations!

Questions?